Documentazione Tecnica – MRS Synthesis Engine

Università degli Studi dell'Aquila – DISIM

Laurea Magistrale in Ingegneria Informatica – Indirizzo ASE

Autore implementazione: documentazione basata sul codice sorgente e sulla Tesi Magistrale di Giovanni Spaziani

Indice

1. Introduzione e Contesto della Tesi
2. Architettura del Sistema
3. Metodologia Formale (dalla Tesi)
4. Implementazione Dettagliata
5. Flusso di Esecuzione Completo
6. Input e Output del Sistema
7. Ragionamento Semantico e LLM
8. Web Server e Interfaccia Utente
9. Test e Verifica
10. Confronto Tesi vs Implementazione
11. Tecnologie Utilizzate
12. Conclusioni e Lavori Futuri

1. Introduzione e Contesto della Tesi

1.1 Problema Affrontato

Nei sistemi multi-robot (MRS – *Multi-Robot Systems*), più agenti autonomi devono cooperare per portare a termine missioni complesse. Un problema fondamentale è garantire che i robot possano

effettivamente **comunicare e coordinare** il loro lavoro in modo corretto, prima ancora di essere dispiegati sul campo.

La tesi affronta questo problema attraverso un processo di **sintesi automatica**: dati i modelli comportamentali di ciascun robot e una descrizione della missione globale, il sistema verifica formalmente se i robot sono **compatibili** tra loro e, in caso affermativo, produce un modello BPMN **arricchito** (cosiddetto *enhanced*) che integra i canali di comunicazione mancanti.

1.2 Dominio Applicativo

Il caso di studio è il **coordinamento di una missione agricola** in un campo trattato con precisione. I robot coinvolti sono:

Robot	Ruolo
Drone	Rileva le zone con erbacce, seleziona il trattore più vicino, coordina le operazioni
Tagliatore	Trattore che taglia le erbacce nel campo; è disponibile in modo condizionale
Scavatore	Effettua lo scavo del terreno dopo il taglio
Seminatore	Semina il terreno preparato dallo Scavatore

Le interazioni tra i robot avvengono tramite messaggi BPMN (es. `weedPosition`, `availability`, `closestTractor`, `fieldReady`, `digDone`, `seedingDone`).

1.3 Contributo Originale

Rispetto al metodo di sintesi formale proposto dalla tesi, l'implementazione aggiunge:

- **LLM Semantic Reasoner**: integrazione con Google Gemini Flash 2.0 (o OpenAI GPT-4o-mini) per la verifica semantica tramite intelligenza artificiale.
- **Web UI interattiva**: interfaccia browser completa per caricare, modificare, visualizzare i modelli BPMN e avviare la sintesi live.
- **Model discovery dinamica**: aggiornamento in tempo reale dei robot disponibili senza riavvio del server.

2. Architettura del Sistema

2.1 Struttura dei Package

```
it.univaq.disim.mrs.synthesis
├── Main.java                          ← Entry point CLI / Web
├── ai/
│   ├── AISemanticMatcher.java         ← Dispatcher semantico (Hybrid / OWL / LLM)
│   └── LLMClient.java                 ← Client HTTP multi-provider (Gemini, OpenAI, Local)
├── analyzer/
│   ├── LocalProcessAnalyzer.java      ← Estrazione metadati I/O dai BPMN locali
│   ├── GlobalChoreographyAnalyzer.java ← Estrazione MessageFlow dalla coreografia globale
│   ├── BehavioralCompatibilityTester.java ← Verifica raggiungibilità e terminazione LTS
│   └── BehavioralStateSimulator.java ← Simulazione stato per contratti
precondizione/effetto
├── core/
│   └── MultiRobotSynthesisEngine.java ← Orchestratore della pipeline completa
├── event/
│   ├── SynthesisEvent.java            ← POJO per eventi live
│   └── SynthesisEventListener.java    ← Interfaccia callback per SSE
├── generator/
│   ├── UnifiedDiagramBuilder.java     ← Costruisce il diagramma BPMN unificato (4 pool)
│   ├── DrawioExporter.java            ← Esporta in formato Draw.io XML
│   └── BpmndiGenerator.java           ← Genera elementi BpmnDI (posizionamento grafico)
├── input/
│   └── UserInputManager.java          ← Carica e valida semantics.json
├── model/
│   ├── DataNode.java                  ← Specifica semantica di un messaggio
│   ├── InteractionMatch.java          ← Coppia produttore-consumatore validata
│   ├── InteractionPlan.java           ← Piano di attivazione (SEQUENCE/PARALLEL/EXCLUSIVE)
│   ├── MissionContext.java            ← Contesto globale della pipeline
│   └── RobotTaskMetadata.java         ← Metadati I/O di un task locale
├── persistence/
│   └── ModelRepository.java           ← Carica/salva BpmnModelInstance da/su file
├── planner/
│   └── InteractionPlanner.java        ← Determina la modalità di attivazione
├── semantic/
│   ├── OntologyManager.java           ← Gerarchia OWL-like da taxonomy.json
│   └── SyntacticCompatibilityChecker.java ← Verifica tipo sintattica (strong/weak)
├── weaver/
│   ├── BpmnFlowWeaver.java            ← Inserisce SendTask/ReceiveTask nei BPMN locali
│   └── BpmnAutoLayout.java            ← Posizionamento automatico degli elementi grafici
└── web/
    └── SynthesisWebServer.java        ← Server HTTP embedded (Java HttpServer)
```

2.2 Grafico delle Dipendenze tra Moduli

```
Main
 └─► MultiRobotSynthesisEngine
       ├─► ModelRepository            (carica/salva .bpmn)
       ├─► OntologyManager            (taxonomy.json → gerarchia tipi)
       ├─► SyntacticCompatibilityChecker (usa OntologyManager)
       ├─► LocalProcessAnalyzer       (estrae metadati da BPMN locali)
       ├─► GlobalChoreographyAnalyzer (estrae MessageFlow)
       ├─► AISemanticMatcher
       │     └─► LLMClient            (Gemini / OpenAI / Local)
       ├─► BehavioralCompatibilityTester (raggiungibilità LTS)
       ├─► BehavioralStateSimulator   (simulazione contratti)
       ├─► UserInputManager           (semantics.json → List<DataNode>)
       ├─► InteractionPlanner         (crea InteractionPlan)
       ├─► BpmnFlowWeaver
       │     └─► BpmnAutoLayout       (posizionamento DI)
       └─► DrawioExporter             (esportazione Draw.io)

SynthesisWebServer
  ├─► MultiRobotSynthesisEngine       (eseguzione pipeline)
  ├─► SynthesisEventListener          (SSE broadcast)
  └─► (index.html via /web/* static)
```

2.3 Modello Dati Centrale

Il `MissionContext` è il POJO che attraversa tutta la pipeline:

```
MissionContext {
  globalChoreography: BpmnModelInstance       ← GlobalMission.bpmn
  robotModels: Map<String, BpmnModelInstance> ← Drone.bpmn, Tagliatore.bpmn, ...
  extractedMetadata: List<RobotTaskMetadata>  ← I/O contracts di ogni task
  validatedMatches: List<InteractionMatch>    ← Interazioni verificate
  diagnostics: List<String>                  ← Errori/avvisi formali
  behavioralNotes: List<String>              ← Avvisi LTS (non bloccanti)
}
```

3. Metodologia Formale

La tesi descrive un metodo automatico in **7 passi** per la sintesi di sistemi multi-robot. Di seguito ogni passo con la sua implementazione.

Passo 1 – Analisi Processi Locali (I/O Contracts)

Obiettivo (tesi): Estrarre da ogni modello BPMN locale i contratti I/O di ciascun task: dato prodotto (`producedData`), dato richiesto (`requiredData`), precondizioni e effetti comportamentali.

Implementato in:

Comportamento reale: Il metodo visita tutti gli elementi `Task` del BPMN, lege le estensioni Camunda (`camunda:outputParameter` , `camunda:inputParameter`) sia nel formato diretto (vecchio) che nel formato `camunda:inputOutput` annidato. Costruisce un `RobotTaskMetadata` per ogni task con:

- `producedDataType` : tipo di dato prodotto (es. `Coordinate`)
- `requiredDataTypes` : set di tipi richiesti (es. `Boolean`)
- `preconditions` : es. `availability_reported=true`
- `effects` : es. `digging_completed=true`

Esempio (Drone):

```
Task_Coordinate → producedDataType: "Coordinate"
Task_SelectClosestTractor → producedDataType: "CoordinateTrattoreVicino"
Task_CheckAvailability → requiredDataTypes: ["Boolean"]
                         preconditions: []
                         effects: ["availability_reported=true"]
```

Passo 2 – Analisi Coreografia Globale

Obiettivo (tesi): Estrarre i `MessageFlow` dalla coreografia globale per identificare quali robot inviano e ricevono quali messaggi.

Implementato in:

Metodi principali:

- `extractGlobalMessages()` : restituisce la lista di tutti i `MessageFlow` del modello.
- `validateStructuralCoherence()` : verifica che esista almeno un StartEvent e un EndEvent, che tutti i nodi siano raggiungibili, che i gateway esclusivi abbiano condizioni booleane complementari, e che eventuali cicli seguano il pattern supportato (timer + gateway booleano complementare). Usa l'algoritmo di Tarjan (SCC) per rilevare i cicli.
- `getParticipantName()` : risolve il nome del partecipante (pool) a partire da un `InteractionNode` .

Esempio: Per `GlobalMission.bpmn` , vengono estratti 6 MessageFlow: `weedPosition` (Drone → Tagliatore), `availability` (Tagliatore → Drone), `closestTractor` (Drone → Tagliatore), `fieldReady` (Tagliatore → Scavatore), `digDone` (Scavatore → Seminatore), `seedingDone` (Seminatore → broadcast).

Passo 3 – Verifica Compatibilità Sintattica

Obiettivo (tesi): Verificare che il tipo di dato prodotto dal mittente sia **compatibile** (identico o sottotipo) con quello atteso dal destinatario, secondo la tassonomia formale.

Implementato in: +

OntologyManager carica `taxonomy.json` in due gerarchie:

- `dataTypes`: tipi di dato (es. `CoordinateTrattoreVicino` → `Coordinate` → `Object`)
- `semanticConcepts`: concetti semantici (es. `work_area_location` → `location`)

Implementa la **sussunzione direzionale** tramite BFS: `produced ⊑ required` sse `produced` è identico a `required` o raggiunge `required` seguendo i supertypes.

Modalità:

- `isStronglyCompatible()`: identità esatta dei tipi base.
- `isWeaklyCompatible()` / `isCompatible()`: sussunzione via gerarchia.

Verifica strutturale campi: `isStructurallyCompatible()` in `SyntacticCompatibilityChecker` verifica che tutti i campi richiesti dal consumatore siano un sottoinsieme di quelli prodotti (ottenuti da `OntologyManager.getFieldsForType()`).

Passo 4 – Verifica Compatibilità Semantica

Obiettivo (tesi): Verificare che le descrizioni semantiche in linguaggio naturale (o i concetti ontologici) del produttore e del consumatore siano **operativamente equivalenti**.

Implementato in:

Tre modalità:

Modalità	Comportamento
`HYBRID`	Prima verifica compatibilità tipo; se sono presenti concetti ontologici, usa `OntologyManager.isSemanticConceptCompatible()`; altrimenti fallback all'LLM.
`ONTOLOGY_STRICT`	Solo ontologia OWL formale; fallisce se mancano i concetti.
`LLM_REASONER`	Bypassa i concetti rigidi, usa il ragionamento neurale in linguaggio naturale.

Passo 5 – Verifica Compatibilità Comportamentale (LTS)

Obiettivo (tesi): Verificare che il processo del consumatore possa **raggiungere** il task che riceve il messaggio (livello A), che da quel task esista sempre un **percorso verso un EndEvent** senza vicoli ciechi (livello B), e che le **precondizioni** del consumatore siano soddisfatte dagli effetti accumulati dal produttore (livello D).

Implementato in: +

`BehavioralCompatibilityTester`:

- `assess()`: verifica il tipo, la raggiungibilità dallo StartEvent (BFS forward) e la terminazione (BFS su tutte le continuazioni, nessun dead-end).
- `prefixPaths()`: enumera tutti i percorsi finiti dallo StartEvent fino al task produttore (con limite `maxPaths=256`).
- `continuationPaths()`: enumera tutti i percorsi finiti dal task consumatore fino all'EndEvent.

`BehavioralStateSimulator.simulate()`: Dato un percorso di task (prefisso + continuazione) e uno stato iniziale (`behavioral-state.json`), esegue la simulazione step-by-step:

1. Verifica che ogni precondizione sia soddisfatta dallo stato corrente.
2. Applica gli effetti al dizionario di stato.
3. Se qualsiasi precondizione è `UNRESOLVED` (chiave non in stato) oppure `FAIL` (valore diverso), la pipeline lo riporta.

Passo 6 – BPMN Flow Weaving

Obiettivo (tesi): Inserire automaticamente `SendTask` e `ReceiveTask` nei modelli BPMN locali, nei punti corretti della sequenza, rispettando l'ordine definito dalla coreografia globale.

Implementato in:

Funzionamento:

1. **Preflight check**: verifica che ogni anchor (task produttore/consumatore) esista nel BPMN locale e abbia il numero corretto di flussi entranti/uscenti prima di modificare qualsiasi modello.
2. **`insertSendTasks()`**: inserisce `SendTask` subito dopo il task produttore, con tre modalità:
    - `SEQUENCE`: sequenza lineare di SendTask.
    - `PARALLEL`: split parallelo (ParallelGateway) + join.
    - `EXCLUSIVE`: split esclusivo (ExclusiveGateway) con condizioni booleane.

3. `insertReceiveTasks()` : inserisce `ReceiveTask` prima del task consumatore, rispettando l'ordine globale (ricalcola l'anchor tramite `effectiveReceiveAnchor()`).
4. `projectAvailabilityRetryControl()` : proietta il ciclo di retry (timer 10s + gateway booleano) dalla coreografia globale nel processo locale del Drone/Tagliatore.
5. `validateLocalModel()` : verifica integrità del modello modificato (ID unici, flussi completi, ogni SendTask/ReceiveTask raggiungibile da Start a End).
6. `BpmnAutoLayout` : posiziona automaticamente gli elementi aggiunti nel canvas BPMN (coordinate X/Y) e sposta i nodi successivi per evitare sovrapposizioni.

Passo 7 – Generazione Output

Implementato in: `ModelRepository` , `UnifiedDiagramBuilder` , `DrawioExporter`

- `ModelRepository.saveModel()` : salva ogni modello BPMN enhanced in `target/output/<Robot>_enhanced.bpmn` .
- `ModelRepository.saveUnifiedDiagram()` : costruisce il BPMN unificato con 4 pool in `target/output/UnifiedMission.bpmn` .
- `DrawioExporter.exportAllToDrawio()` : genera i file `.drawio` per visualizzazione in Draw.io.

4. Implementazione Dettagliata

4.1 `MultiRobotSynthesisEngine` – Orchestratore

Responsabilità: Coordina l'intera pipeline in sequenza, emette eventi SSE a ogni passo, gestisce la propagazione degli effetti di stato tra le interazioni.

Metodo centrale: `runSynthesisPipeline()`

```
1. validateInputs()          → controlla esistenza di tutti i file
2. clearPreviousOutputs()    → cancella target/output/*.bpmn e *.drawio
3. repository.loadModel()    → carica GlobalMission.bpmn
4. ontologyManager.loadOntology() → carica taxonomy.json
5. inputManager.loadSemanticSpecifications() → carica semantics.json → List<DataNode>
6. loadInitialBehavioralState() → carica behavioral-state.json → Map<String,String>
7. Per ogni robot locale:
   localAnalyzer.analyzeLocalModels() → List<RobotTaskMetadata>
8. globalAnalyzer.validateStructuralCoherence() → List<String> issues
9. globalAnalyzer.extractGlobalMessages() → List<MessageFlow>
10. Per ogni MessageFlow:
    resolveMessageFlow() → verifica sintattica + comportamentale + semantica
11. planner.buildInteractionPlans() → List<InteractionPlan>
12. weaver.weaveInteractions() → modifica BPMN in-memory
13. repository.saveModel() × N → <Robot>_enhanced.bpmn
14. repository.saveUnifiedDiagram() → UnifiedMission.bpmn
15. drawioExporter.exportAllToDrawio() → *.drawio
```

Metodo `resolveMessageFlow()` – per ogni MessageFlow della coreografia globale:

1. Trova la specifica semantica in `semantics.json` (match per `messageName`).
2. Verifica che il robot sorgente coincida con `robotId` in semantics.
3. Trova il task produttore nei metadati locali.
4. Verifica compatibilità sintattica tipo produttore → tipo messaggio.
5. Verifica compatibilità strutturale campi (se presenti).
6. Calcola lo stato BFS precedente accumulando gli effetti delle interazioni già validate.
7. Verifica comportamentale (prefissi produttore, raggiungibilità consumatore, continuazioni).
8. Verifica semantica tramite `AISemanticMatcher`.
9. Aggiunge l'`InteractionMatch` ai match validati del contesto.

Propagazione stato: `stateWithPriorInteractionEffects()` accumula gli effetti dichiarati da ogni task produttore già validato, aggiornando lo stato per la verifica comportamentale del prossimo messaggio.

4.2 `LocalProcessAnalyzer`

Analizza le estensioni BPMN di ogni `Task`. Supporta due formati:

Formato diretto (legacy):

```xml
<bpmn:extensionElements>
  <camunda:outputParameter name="producedData">Coordinate</camunda:outputParameter>
  <camunda:inputParameter name="precondition">mission_started=true</camunda:inputParameter>
  <camunda:outputParameter name="effect">weed_detected=true</camunda:outputParameter>
</bpmn:extensionElements>
```

Formato `camunda:inputOutput` (standard):

```xml
<bpmn:extensionElements>
  <camunda:inputOutput>
    <camunda:outputParameter name="producedData">Boolean</camunda:outputParameter>
    <camunda:outputParameter name="effect">digging_completed=true</camunda:outputParameter>
  </camunda:inputOutput>
</bpmn:extensionElements>
```

> **Nota:** Se un task ha più di un `producedData`, il metodo lancia `IllegalArgumentException`. Un task può avere zero producedData (task puramente consumatore).

4.3 `GlobalChoreographyAnalyzer`

`validateStructuralCoherence()` esegue controlli formali:

- Presenza di StartEvent e EndEvent.
- Raggiungibilità di tutti i FlowNode dallo Start (BFS forward).
- Raggiungibilità di ogni FlowNode verso almeno un End (BFS backward).
- Per ExclusiveGateway: almeno 2 rami, condizioni `${var == true/false}` complementari, variabile dichiarata come messageName con un MessageFlow associato.
- Per cicli (SCC via Tarjan): accettati solo se contengono un `IntermediateCatchEvent` con `timeDuration` e un ExclusiveGateway a condizioni booleane complementari. Altrimenti l'issue è segnalato ma non blocca la pipeline (è un warning).

Algoritmo di Tarjan (SCC) implementato in `strongConnect()`: trova le componenti fortemente connesse del grafo del flusso globale per rilevare i cicli.

4.4 `OntologyManager`

Implementa la gerarchia di tipi OWL-like da `taxonomy.json`.

Formato supportato:

```
json
{
  "dataTypes": {
    "CoordinateTrattoreVicino": ["Coordinate"],
    "Coordinate": ["Object"],
    "Boolean": ["Object"]
  },
  "semanticConcepts": {
    "work_area_location": ["location"],
    "selected_tractor_location": ["location"],
    "robot_availability": ["state"],
    "field_ready_for_digging": ["state"],
    "digging_completed": ["state"],
    "mission_completed": ["state"]
  }
}
```

Sussunzione direzionale `isSubsumedBy(child, parent, hierarchy)`**:** BFS che parte da `child` seguendo i supertypes. Se raggiunge `parent`, ritorna `true`. La relazione è irreflessa (un tipo non è mai suo proprio supertipo nei controlli di compatibilità, anche se ovviamente `child.equals(parent)` è caso base).

Rilevamento cicli `detectCycles()`**:** per ogni concetto, verifica che non sia raggiungibile da se stesso tramite i supertypes. Lancia `IllegalArgumentException` se rilevato un ciclo, perché una gerarchia ciclica renderebbe il ragionamento non terminante.

4.5 `BehavioralCompatibilityTester`

`assess()`**:** Verifica livello A (raggiungibilità) e B (terminazione):

- BFS forward da ogni StartEvent verso il task consumatore.
- DFS ricorsivo con memoizzazione (`assessTermination()`) per verificare che ogni continuazione raggiunga un EndEvent senza dead-end.

`prefixPaths()` **e** `continuationPaths()`**:** Enumerazione esaustiva di percorsi con limite `maxPaths=256`. Un percorso che supera questo limite causa un risultato `UNRESOLVED` con diagnostica.

Rappresentazione stato interno:

- `Status.PASS` : tutto verificato.
- `Status.FAIL` : incompatibilità rilevata.
- `Status.UNRESOLVED` : verifica non completabile per dati mancanti.

4.6 `BehavioralStateSimulator`

Simulatore deterministico di contratti:

```
java
simulate(List<RobotTaskMetadata> path, Map<String,String> initialState)
```

Algoritmo:

1. Copia lo stato iniziale in un `LinkedHashMap<String,String>` .
2. Per ogni task nel percorso:
    - Valida internamente le precondizioni (no contraddizioni).
    - Verifica ogni precondizione: `state.get(key).equals(expectedValue)` . Se la chiave non esiste: `UNRESOLVED` . Se il valore differisce: `FAIL` .
    - Applica gli effetti: `state.put(key, value)` .
3. Se nessun contratto è dichiarato nel percorso: `UNRESOLVED` (non si può provare senza contratti espliciti).

Formato letterale: `chiave=valore` (es. `digging_completed=true` , `mission_started=true`).

5. Flusso di Esecuzione Completo

5.1 Modalità CLI

```
mvn exec:java -Dexec.args="src/main/resources/bpmn/GlobalMission.bpmn
src/main/resources/bpmn taxonomy.json semantics.json"
```

```
Main.main(args)
  → discoverGlobalModel()          [se non passato esplicitamente]
  → discoverLocalModels()          [filtra i BPMN locali dal global]
  → new MultiRobotSynthesisEngine()
  → runSynthesisPipeline(locals, global, ontology, semantics, strict=false)
     → [Pipeline 7 passi]
  → log "Synthesis result: 6 validated, 0 diagnostics"
```

Auto-discovery Global: `Main.isGlobalChoreography()` identifica il file globale verificando che il BPMN contenga almeno un `Collaboration` con almeno 2 `Participant` e almeno 1 `MessageFlow`. Se trovato esattamente 1 candidato, lo usa; altrimenti richiede il path esplicito.

5.2 Modalità Web Server

```
Main.main(["--web"])
  → SynthesisWebServer.main(args)
     → new SynthesisWebServer()
     → server.start()     [HttpServer sulla porta 8080]
     → Registra handler:
        GET  /                  → index.html
        GET  /web/*             → static files
        GET  /api/status        → {"status":"UP", ...}
        GET  /api/events        → SSE stream
        POST /api/synthesize    → avvia pipeline in background thread
        GET  /api/models        → lista BPMN disponibili
        GET  /api/inputs        → metadata completi (global + robots + configs)
        GET  /api/file-content  → contenuto file (JSON/XML)
        POST /api/file-content  → salva file con validazione
        POST /api/upload        → carica file binario con X-Target header
        DELETE /api/models?name= → cancella robot BPMN
        POST /api/reset-inputs  → ripristina .defaults/
        GET  /api/bpmn/{file}   → download BPMN (output o bpmn dir)
        GET  /api/drawio/{file} → download Draw.io
```

5.3 Sequenza di una Sintesi Live

```
Browser → POST /api/synthesize?delay=850&matcher=LLM_REASONER&apiKey=AIzaSy...
Server  → Avvia thread background
        → engine.setEventListener(sseListener)
        → engine.runSynthesisPipeline(...)
           → emit(PIPELINE_STARTED)   → SSE broadcast → Browser log
           → emit(STEP_STARTED, 1)    → Browser: step 1 attivo
           → emit(ROBOT_LOADED, Drone)
           → emit(GLOBAL_MESSAGE_DISCOVERED, weedPosition)
           → emit(SYNTACTIC_CHECK, OK: Coordinate → Coordinate)
           → emit(BEHAVIORAL_CHECK, deadlock-free)
           → emit(SEMANTIC_CHECK, Gemini: COMPATIBILE 0.98)
           ... × 6 interazioni
           → emit(PIPELINE_COMPLETED)
Server  → Salva _enhanced.bpmn e UnifiedMission.bpmn
Browser → Carica UnifiedMission.bpmn nel viewer BPMN-JS
```

6. Input e Output del Sistema

6.1 `semantics.json` – Specifica Semantica delle Interazioni

Formato: array JSON di oggetti `DataNode`.

```
json
[
  {
    "robotId": "Drone",                    // Robot produttore
    "taskId": "Task_Coordinate",           // Task produttore (ID BPMN)
    "messageName": "weedPosition",         // Nome MessageFlow globale
    "dataType": "Coordinate",              // Tipo prodotto
    "semanticDescription": "Posizione geografica...",  // Descrizione NL per LLM
    "semanticConcept": "work_area_location",          // Concetto OWL produttore
    "receiverTaskId": "Task_Movimento",    // Task consumatore (ID BPMN)
    "receiverDataType": "Coordinate",      // Tipo richiesto dal consumatore
    "receiverSemanticDescription": "Posizione della zona di lavoro...",
    "receiverSemanticConcept": "work_area_location"
  },
  ...
]
```

Campi opzionali: `receiverTaskId` può essere `null` per messaggi broadcast; `syntacticCompatibility` può essere `"STRONG"` per richiedere identità di tipo invece di sussunzione.

Le 6 specifiche del caso di studio:

1. `weedPosition`: Drone (Task_Coordinate) → Tagliatore (Task_Movimento), tipo `Coordinate`
2. `availability`: Tagliatore (Task_Autovalutazione) → Drone (Task_CheckAvailability), tipo `Boolean`
3. `closestTractor`: Drone (Task_SelectClosestTractor) → Tagliatore (Task_Movimento), tipo `CoordinateTrattoreVicino`
4. `fieldReady`: Tagliatore (Task_Taglio) → Scavatore (Task_Posizionamento), tipo `Boolean`
5. `digDone`: Scavatore (Task_Scavo) → Seminatore (Task_Carica), tipo `Boolean`
6. `seedingDone`: Seminatore (Task_Semina) → broadcast, tipo `Boolean`

6.2 `taxonomy.json` – Ontologia e Tassonomia

```
json
{
  "dataTypes": {
    "CoordinateTrattoreVicino": ["Coordinate"],
    "Coordinate": ["Object"],
    "Boolean": ["Object"],
    "Object": []
  },
  "semanticConcepts": {
    "work_area_location": ["location"],
    "selected_tractor_location": ["location"],
    "location": [],
    "robot_availability": ["state"],
    "field_ready_for_digging": ["state"],
    "digging_completed": ["state"],
    "mission_completed": ["state"],
    "state": []
  }
}
```

Significato: `CoordinateTrattoreVicino ⊑ Coordinate ⊑ Object`. Perciò un task che richiede `Coordinate` può accettare `CoordinateTrattoreVicino` (sussunzione). La relazione è direzionale: il contrario non vale.

6.3 `behavioral-state.json` – Stato Iniziale LTS

```
json
{
  "description": "Explicit initial facts for the thesis case study",
  "initialState": {
    "mission_started": "true"
  }
}
```

Formato: oggetto `initialState` con coppie `chiave: valore_scalare`. Lo stato viene arricchito durante la pipeline da `stateWithPriorInteractionEffects()` con gli effetti dichiarati nei task già verificati.

6.4 BPMN Locali – Formato Esteso

Ogni robot locale è un file BPMN con estensioni Camunda sui task:

```
xml
<bpmn:task id="Task_Taglio" name="Taglio Erbacce">
  <bpmn:extensionElements>
    <camunda:inputOutput>
      <camunda:inputParameter name="requiredData">Coordinate</camunda:inputParameter>
      <camunda:inputParameter
name="precondition">availability_reported=true</camunda:inputParameter>
      <camunda:outputParameter name="producedData">Boolean</camunda:outputParameter>
      <camunda:outputParameter name="effect">field_ready=true</camunda:outputParameter>
    </camunda:inputOutput>
  </bpmn:extensionElements>
</bpmn:task>
```

6.5 Output Prodotti

File	Percorso	Contenuto
`Drone_enhanced.bpmn`	`target/output/`	BPMN Drone con SendTask/ReceiveTask inseriti
`Tagliatore_enhanced.bpmn`	`target/output/`	BPMN Tagliatore con retry control proiettato
`Scavatore_enhanced.bpmn`	`target/output/`	BPMN Scavatore con ReceiveTask/SendTask
`Seminatore_enhanced.bpmn`	`target/output/`	BPMN Seminatore con ReceiveTask/SendTask
`UnifiedMission.bpmn`	`target/output/`	Collaborazione BPMN con 4 pool separati
`*.drawio`	`target/output/`	Diagrammi Draw.io per ogni enhanced model

7. Ragionamento Semantico e LLM

7.1 I Tre Matcher

HYBRID (default)

```
isSemanticMatch(producedType, prodDesc, requiredType, reqDesc, prodConcept, reqConcept)
  1. isTypeCompatible(producedType, requiredType)  → usa OntologyManager
     → FAIL se tipi non compatibili
  2. Se concetti ontologici presenti:
     → ontologyManager.isSemanticConceptCompatible(prodConcept, reqConcept)
     → usa gerarchia semanticConcepts
  3. Se concetti assenti o incompatibili:
     → llmClient.askAIForCompatibility(...)  [fallback LLM]
```

ONTOLOGY_STRICT

```
  1. isTypeCompatible()
  2. Se concetti mancanti → FAIL immediato
  3. ontologyManager.isSemanticConceptCompatible()
```

LLM_REASONER

```
1. isTypeCompatible()
2. llmClient.askAIForCompatibility() → sempre, bypassa ontologia
```

7.2 `LLMClient` – Cascata di Provider

```
askAIForCompatibility(producedType, prodDesc, requiredType, reqDesc)
  1. customJudge     → solo nei test (mock)
  2. Gemini Flash    → resolveApiKey(GEMINI_API_KEY, GOOGLE_API_KEY)
                       → chiamata HTTP POST con payload JSON
                       → parsing risposta JSON: {compatible, confidence, reasoning}
  3. OpenAI GPT-4o   → resolveApiKey(OPENAI_API_KEY)
  4. Local Reasoner  → pattern matching NL su dominio agricolo
```

7.3 `resolveApiKey()` – Cascata di Fonti

```
1. explicitApiKey    (inviata dalla Web UI)
2. System.getProperty(gemini.api.key)
3. System.getenv(GEMINI_API_KEY) / GOOGLE_API_KEY
4. File "gemini.key" nella directory corrente
5. File ".env" nella directory corrente (formato KEY=value)
```

7.4 Dynamic Model Discovery

Prima di chiamare l'API Gemini, `queryGoogleModelService()`:

1. Chiama `GET /v1beta/models?key=...` e `GET /v1/models?key=...`.
2. Filtra i modelli che supportano `generateContent`.
3. Esclude modelli TTS, audio, embed, image, vision-preview, realtime.
4. Assegna un punteggio a ogni modello (`scoreModel()`):
   - `gemini-2.0-flash` (non lite) → 100
   - `gemini-1.5-flash-latest` → 95
   - `gemini-1.5-flash-001/002` → 92
   - `gemini-2.0-flash-lite` → 85
   - `gemini-pro` → 75
5. Usa il modello col punteggio più alto; in caso di fallimento, prova il successivo.

6. Memorizza la URL funzionante in `cachedGeminiUrl` per le chiamate successive.

7.5 Prompt Engineering

Il prompt inviato a Gemini è strutturato in questo modo:

```
You are an expert AI semantic reasoner for a Multi-Robot System (MRS) in precision
agriculture.
Determine whether the PRODUCER's output data and the CONSUMER's input requirement represent
matching,
compatible operational concepts in the robotic mission workflow.

Robotics Domain Knowledge:
1. Drone detects weeds coordinates → Cutter tractor work area location: COMPATIBLE
2. Cutter tractor availability state → Drone checks availability: COMPATIBLE
3. Drone selected closest tractor → Cutter receives assignment: COMPATIBLE
4. Cutter finished / field ready signal → Excavator ready to dig: COMPATIBLE
5. Excavator digging completed → Seeder ready to seed: COMPATIBLE
6. Seeder completed notification → Mission completed broadcast: COMPATIBLE

Pair to evaluate:
- PRODUCER data: type=Coordinate, description="Posizione geografica della zona di
interesse..."
- CONSUMER requirement: type=Coordinate, description="Posizione della zona di lavoro da
raggiungere..."

Respond ONLY with JSON (no markdown):
{"compatible": true, "confidence": 0.98, "reasoning": "Spiegazione in italiano"}
```

Parametri: `temperature=0.0` (risposta deterministica), `maxOutputTokens=512`.

7.6 Local Semantic Reasoner (Fallback)

Quando né Gemini né OpenAI sono disponibili, `evaluateLocalSemanticReasoning()` usa pattern matching sul dominio agricolo:

```
java
weedCoords     = p.contains("erbacce"/"zona") && r.contains("lavoro"/"taglio")
availability   = p.contains("disponibil") && r.contains("missione")
tractorSelect  = p.contains("trattore"/"vicino") && r.contains("assegnat")
fieldReady     = p.contains("pronto"/"scavo") && r.contains("terreno")
digDone        = p.contains("scavo"/"completament") && r.contains("semina")
seedingDone    = p.contains("semina") && r.contains("completament"/"missione")
```

Se uno dei pattern corrisponde: `Confidence: 0.97`. Altrimenti: `FALSE`.

> **Attenzione:** Il reasoner locale è strettamente hardcodato per il dominio agricolo del caso di studio. Per un dominio diverso, restituirebbe `FALSE` per tutte le interazioni se non adattato.

8. Web Server e Interfaccia Utente

8.1 Endpoint REST Completi

Metodo	URL	Descrizione
`GET`	`/`	Redirect a `/web/index.html`
`GET`	`/web/*`	File statici (`index.html`, CSS, JS)
`GET`	`/api/status`	JSON: `{status, version, port, activeSseClients, historyEventCount}`
`GET`	`/api/events`	SSE stream (`text/event-stream`); invia la storia degli eventi + live
`POST`	`/api/synthesize`	Avvia pipeline. Params: `delay` (ms), `matcher` (HYBRID/ONTOLOGY_STRICT/LLM_REASONER), `apiKey`
`GET`	`/api/models`	Lista tutti i `.bpmn` nella directory BPMN
`GET`	`/api/inputs`	Metadata completi: global mission, lista robots, config (semantics/taxonomy/behavioral)
`GET`	`/api/file-content?target=&file=`	Contenuto testuale di un file
`POST`	`/api/file-content`	Salva file con validazione JSON o XML BPMN
`POST`	`/api/upload`	Upload binario; `X-Target` header: global/robot/semantics/taxonomy/behavioral
`DELETE`	`/api/models?name=`	Cancella robot BPMN (protegge GlobalMission.bpmn)
`POST`	`/api/reset-inputs`	Copia i file da `.defaults/` sovrascrivendo quelli correnti
`GET`	`/api/bpmn/{file}`	Download BPMN (cerca prima in `target/output/`, poi in `bpmn/`)
`GET`	`/api/drawio/{file}`	Download file Draw.io

8.2 SSE Event Streaming

Il server mantiene una `CopyOnWriteArrayList<HttpExchange>` di client SSE connessi (`sseClients`). Ogni evento della pipeline viene:

1. Serializzato come JSON.
2. Aggiunto alla `eventHistory` (per i client che si connettono tardi).
3. Inviato a tutti i client attivi nel formato SSE: `data: {json}\n\n`.

Tipi di eventi SSE:

```
PIPELINE_STARTED, STEP_STARTED, ROBOT_LOADED,
GLOBAL_MESSAGE_DISCOVERED, SYNTACTIC_CHECK, BEHAVIORAL_CHECK,
SEMANTIC_CHECK, PLAN_CREATED, WEAVING_TASK, PIPELINE_COMPLETED,
DIAGNOSTIC
```

8.3 Validazione File nel Server

JSON: `mapper.readTree(content)` via Jackson – se lancia eccezione, HTTP 400.

BPMN/XML:

```
java
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
dbf.newDocumentBuilder().parse(new InputSource(new StringReader(content)));
```

La feature `disallow-doctype-decl` previene XXE injection.

8.4 Interfaccia Utente (`index.html`)

Stack tecnologico frontend:

- **Tailwind CSS** (CDN): utility-first styling.
- **bpmn-js 17.11.1** (CDN): viewer BPMN interattivo.

Struttura UI:

- **Navbar** in alto: selettore modalità (Demo/Turbo), selettore modello attivo, pulsante "Modifica BPMN", download BPMN/Draw.io, pulsante "Avvia Sintesi".
- **Pipeline Stepper Banner**: 4 badge colorati (Sintassi → Semantica → Comportamento → Weaving) aggiornati live dagli eventi SSE.
- **Left Sidebar**: Gestione input (coreografia globale, robot locali, semantics, taxonomy, behavioral, selettore matcher, API key).
- **Main Content** (3 tab):
    - **Canvas**: viewer BPMN-JS interattivo con zoom/pan.

    - **Matrice**: tabella HTML statica delle 6 interazioni verificate.
    - **Log Terminal**: stream SSE con colori per tipo di evento.

Funzioni JavaScript principali:

- `loadInputsMetadata()`: chiama `/api/inputs`, popola dinamicamente la lista robot e le card dei config.
- `startSynthesis()`: POST `/api/synthesize` con i parametri dalla UI.
- `initSse()`: apre `EventSource('/api/events')`, chiama `handleLiveEvent()` per ogni evento.
- `openFileEditor(target, file, desc)`: carica il contenuto del file via `/api/file-content`, apre il modal.
- `saveModalFileContent()`: POST `/api/file-content` col contenuto modificato, poi aggiorna la lista e il viewer.
- `formatModalContent()`: formatta JSON o XML nel textarea del modal editor.
- `editCurrentActiveModel()`: apre l'editor per il modello selezionato nel dropdown superiore.
- `loadBpmnFile(fileName)`: fetch del BPMN e rendering in bpmn-js.
- `triggerTargetUpload(target)` + `handleTargetedUpload()`: carica file binari via `/api/upload`.
- `deleteRobotModel(name)`: DELETE `/api/models?name=`.
- `resetToDefaultInputs()`: POST `/api/reset-inputs`.

9. Test e Verifica

9.1 Suite di Test (63 test, tutti PASS)

Classe di Test	Test	Comportamenti Verificati
`SyntacticCompatibilityCheckerTest`	~12	Compatibilità strong/weak, subsumption, campi strutturali
`OntologyManagerTest`	~8	Caricamento taxonomy.json, subsumption transitiva, rilevamento cicli
`BehavioralCompatibilityTesterTest`	~10	Raggiungibilità, dead-end, enumerazione percorsi
`BehavioralStateSimulatorTest`	~8	Precondizioni soddisfatte, violate, UNRESOLVED
`MultiRobotSynthesisEngineTest`	~10	Pipeline end-to-end con i 4 robot del caso di studio
`LLMClientTest`	~5	Custom judge mock, provider fallback, parsing risposta
`SynthesisWebServerTest`	~5	Avvio server, endpoint status, file content
`UserInputManagerCompatibilityTest`	~3	Caricamento semantics.json, validazione campi
`MainInputDiscoveryTest`	~2	Discovery global model, discovery local models

9.2 Test di Integrazione End-to-End

`MultiRobotSynthesisEngineTest` esegue la pipeline completa con i file reali:

- 4 robot (`Drone.bpmn`, `Tagliatore.bpmn`, `Scavatore.bpmn`, `Seminatore.bpmn`)
- `GlobalMission.bpmn`, `taxonomy.json`, `semantics.json`, `behavioral-state.json`
- Verifica che siano prodotti esattamente 6 `InteractionMatch` validati.
- Verifica che `diagnostics` sia vuota in caso di input corretti.
- Verifica che i file `_enhanced.bpmn` e `UnifiedMission.bpmn` siano generati in `target/output/`.

9.3 Aspetti Non Testati o Parzialmente Testati

Aspetto	Stato
Test della Web UI (endpoint HTTP reali)	Parziale (solo `/api/status`)
Test di upload file via `/api/upload`	Non presente
Test del reset `/api/reset-inputs`	Non presente
Test del `BpmnAutoLayout` (posizionamento DI)	Non presente
Test del `DrawioExporter`	Non presente
Test del `UnifiedDiagramBuilder` (pool unificato)	Non presente
Test con dominio diverso dall'agricoltura	Non presente
Test del ciclo di retry proiettato dal Weaver	Parziale

10. Confronto Tesi vs Implementazione

Step Metodologico (Tesi)	Previsto	Implementato	Note
Passo 1: Analisi processi locali (I/O contracts)	✅	✅	Completo; supporta 2 formati BPMN extension
Passo 2: Analisi coreografia globale	✅	✅	Include validazione strutturale (SCC, gateway, cicli)
Passo 3a: Compatibilità sintattica tipo	✅	✅	Strong (identità) e Weak (sussunzione OWL)
Passo 3b: Compatibilità strutturale campi	✅	✅	Verifica sottoinsieme di campi richiesti
Passo 4: Compatibilità semantica	✅ OWL	✅ OWL + LLM	**Estensione rispetto alla tesi:** aggiunto LLM Reasoner (Gemini Flash, OpenAI) non previsto dalla metodologia originale
Passo 5a: Raggiungibilità consumer da Start (Livello A)	✅	✅	BFS forward
Passo 5b: Terminazione verso End (Livello B)	✅	✅	DFS con memoizzazione
Passo 5c: Verifica prefissi/continuazioni LTS (Livello C/D)	✅	✅	Enumerazione percorsi con limite 256
Passo 5d: Simulazione stato (Livello D)	✅	✅	Contratti precondizione/effetto su percorso concatenato
Passo 6: BPMN Flow Weaving	✅	✅	Modalità SEQUENCE, PARALLEL, EXCLUSIVE
Passo 6b: Proiezione retry locale	✅ (implicito)	✅	Proiezione del ciclo availability retry dal global al local
Passo 7: Output BPMN enhanced	✅	✅	_enhanced.bpmn, UnifiedMission.bpmn, Draw.io
Web UI	❌ non prevista	✅	Estensione significativa rispetto alla tesi
API REST	❌ non prevista	✅	Estensione
SSE eventi live	❌ non prevista	✅	Estensione didattica

10.1 Discrepanze Rilevate

1. Local Reasoner hardcodato per il dominio agricolo Il fallback locale in `LLMClient.evaluateLocalSemanticReasoning()` usa pattern matching testuale (es. `p.contains("erbacce")`) specifico per il caso di studio. Se applicato a un dominio diverso senza una chiave API Gemini valida, restituirebbe `FALSE` per tutte le interazioni, bloccando la pipeline. La tesi non tratta questo aspetto.

2. `receiverTaskId: null` per `seedingDone` Nel `semantics.json`, il messaggio `seedingDone` ha `receiverTaskId: null`, il che fa sì che l'engine lo tratti come broadcast verso `targetRobot="Broadcast"`. La logica di verifica è leggermente diversa (non si cerca un consumer specifico), ma il risultato è corretto per il caso di studio.

3. Limite `maxPaths=256` nella simulazione LTS La tesi non specifica un limite esplicito al numero di percorsi enumerati. Il codice introduce un limite di sicurezza (256 combinazioni prefisso × continuazione) per evitare explosion esponenziale. Se superato, l'interazione risulta `UNRESOLVED` anziché `FAIL`, il che è conservativo ma potrebbe non corrispondere al comportamento atteso dalla tesi per grafi molto complessi.

4. `validateStructuralCoherence()` come warning, non blocking La verifica strutturale della coreografia globale (`GlobalChoreographyAnalyzer`) produce una lista di `issues` che vengono aggiunte alle `diagnostics` del contesto, ma **non bloccano** la pipeline. La tesi potrebbe richiedere un fallimento immediato su coreografia malformata.

11. Tecnologie Utilizzate

Libreria / Framework	Versione	Ruolo nel Progetto
Camunda BPMN Model API	7.20.0	Parsing, navigazione e modifica di file BPMN/XML in memoria. Usata da LocalProcessAnalyzer, GlobalChoreographyAnalyzer, BpmnFlowWeaver, ModelRepository, UnifiedDiagramBuilder.
Jackson Databind	2.15.2	Parsing e serializzazione JSON: caricamento di taxonomy.json, semantics.json, behavioral-state.json, payload REST, risposte LLM.
SLF4J Simple	2.0.7	Logging strutturato su stdout. Ogni classe usa `LoggerFactory.getLogger(ClassName.class)`.
JUnit Jupiter 5	5.10.0	Framework di test unitari. Annotazioni `@Test`, `@BeforeEach`, `@ParameterizedTest`.
Java HttpClient (JDK 11+)	built-in	Client HTTP per le chiamate all'API Gemini Flash e OpenAI GPT-4o-mini in `LLMClient`.
Java HttpServer (JDK 11+)	built-in	Web server embedded per la modalità `--web`. Nessuna dipendenza Spring.
Maven	3.x	Build system: compilazione (`maven-compiler-plugin 3.11.0`), test (`maven-surefire-plugin 3.2.5`), esecuzione (`exec-maven-plugin 3.3.0`).
Tailwind CSS	CDN	Styling della Web UI tramite classi utility.
bpmn-js	17.11.1	Viewer BPMN interattivo nel browser. Renderizza i file BPMN via `BpmnNavigatedViewer`.

Configurazione Maven:

- `maven.compiler.release=11`: target Java 11+.
- `useModulePath=false` in Surefire: necessario per evitare conflitti col module path di Java 11+ nella suite di test.
- Main class: `it.univaq.disim.mrs.synthesis.Main`.

12. Conclusioni e Lavori Futuri

12.1 Cosa è Stato Raggiunto

Il progetto implementa fedelmente la metodologia formale descritta nella tesi (7 passi di sintesi), applicata al caso di studio reale dei 4 robot agricoli. I 63 test unitari e di integrazione confermano la correttezza del nucleo formale. L'aggiunta della Web UI interattiva e del LLM Reasoner (Google Gemini Flash 2.0) costituiscono contributi originali significativi rispetto alla tesi, che rendono il sistema:

- **Dimostrabile** in tempo reale (SSE live pipeline).
- **Configurabile** senza ricompilazione (upload file, edit in-browser).
- **Estensibile** a nuovi domini semantici (tramite LLM, non solo ontologia hardcodata).

12.2 Limitazioni Attuali

1. Il Local Semantic Reasoner è hardcodato per il dominio agricolo.
2. Il limite di 256 percorsi LTS potrebbe essere insufficiente per sistemi con molti stati.
3. La Web UI non ha autenticazione (adeguata per demo accademiche, non per produzione).
4. Il file `behavioral-state.json` richiede configurazione manuale dei fatti iniziali.

12.3 Possibili Estensioni

- **Generalizzazione del Local Reasoner**: sostituire i pattern hardcodati con un motore di regole configurabile (es. file `domain-rules.json`).
- **Supporto OWL full**: integrazione di un reasoner OWL reale (es. HermiT/OWL API) in sostituzione della gerarchia semplificata attuale.
- **Multi-tenant Web UI**: autenticazione e sessioni per più utenti concorrenti.
- **Generazione automatica di behavioral-state.json**: inferenza dello stato iniziale dai BPMN locali.
- **Supporto a più coreografie globali**: gestione di missioni con più fasi orchestrate.

Documentazione generata il 09 Novembre 2026 – basata sull'analisi diretta del codice sorgente e sulla Tesi Magistrale di Giovanni Spaziani (UNIVAQ DISIM).